Repair-Based Genetic Algorithm for Course Timetabling under Lecturer Availability Constraints

Slamet Riyadi (F5522630001), Muhammad Adi Agum (F5522630002), Akbar Tri Wicaksono (F5522630003), I Ketut Divta Suryawan (F5522630004)

Study Program of Information Technology (S2), Postgraduate Program — Kecerdasan Komputasional, 2026

Abstract

Course timetabling must satisfy lecturer availability, room usage, student-group conflicts, and room capacity at the same time; as lecturer availability tightens, randomly generated schedules are almost always infeasible, and a genetic algorithm (GA) may spend most of its search on infeasible regions. This study asks a controlled question: at an identical budget of objective evaluations, does repairing hard-constraint violations after crossover and mutation help a GA find feasible timetables more often, and what does that repair cost? We compare three methods — deterministic greedy construction, a standard GA, and the same GA augmented with a repair operator that moves a violating session to a valid room–slot pair with the smallest added preference penalty — on nine synthetic instances (30, 60 and 90 sessions; three lecturer availability patterns) that are feasible by construction, and on twelve public ITC2007 curriculum-based benchmark instances. Every objective evaluation is charged to a common budget of 10,000 per run, including each candidate examined by repair, and 20 seeds are used per stochastic method per instance. Repair raised the share of instances solved to feasibility from 10/21 (standard GA) to 19/21 (GA with repair) and lowered the median hard-violation count from 1 to 0, at the price of 67% of the evaluation budget and 3.1× fewer completed generations. The advantage is concentrated on the public benchmark instances, where room occupancy is high and the deterministic constructor never succeeds; on the controlled availability ladder, where every method already found a feasible timetable, repair instead lowered the preference penalty and left feasibility unchanged. The results give a budget-normalised, reproducible account of when constraint repair pays off in course timetabling and when it merely consumes search capacity.

Keywords: course timetabling; genetic algorithm; constraint repair; evaluation budget; lecturer availability; feasibility

1. Introduction

University course timetabling assigns each required class session to a room and a time slot so that no lecturer, student group or room is used twice in the same slot, room capacity is respected, and lecturers are not scheduled when they are unavailable. The problem is NP-hard in general, and practical instances combine hundreds of sessions with tightly constrained lecturer calendars. Search-based methods — local search, tabu search, simulated annealing, matheuristics and evolutionary algorithms — are therefore the standard toolkit, and the curriculum-based course timetabling (CB-CTT) track of the second International Timetabling Competition (ITC2007) remains the reference benchmark family (Bettinelli et al., 2015; Müller, 2009; Di Gaspero & Schaerf, 2003; Geiger, 2010).

Genetic algorithms are attractive for timetabling because a timetable can be encoded directly as one room–slot pair per session, and crossover and mutation can recombine partial assignments. The same encoding, however, makes offspring fragile: uniform crossover between two feasible parents typically produces a child that violates many hard constraints, because the genes inherited from different parents compete for the same lecturers, groups and rooms. A GA without constraint handling therefore spends its budget scoring infeasible schedules. Two families of remedies are common in the evolutionary literature: penalty functions that keep infeasible solutions in the population but rank them with a penalty term, and repair operators that transform an infeasible candidate into a feasible (or less infeasible) one before evaluation (Coello Coello, 2002). Recent constrained evolutionary algorithms continue to use structure-aware seeding and projection-based repair when the feasible region is narrow (Zhang et al., 2026; Rao & Jia, 2026; Zhao et al., 2026).

Repair is not free. Every candidate move that the operator examines is an objective evaluation that the algorithm could otherwise have spent on selection and variation. Studies of metaheuristics for educational timetabling usually report a time limit or an iteration count, and papers on constraint handling usually report solution quality against a fixed number of generations; neither convention makes the cost of constraint handling explicit. As a consequence it is difficult to answer a practical question: if the total number of objective evaluations is fixed, does repair improve the quality and feasibility of the schedules found, or does it consume the budget that would have produced a better search?

This paper addresses that question for the specific setting named in our course brief: GA with repair for course timetabling under lecturer availability constraints. Our contribution is deliberately narrow and empirical rather than architectural. We (i) implement a GA in which one gene is a room–slot pair and the objective is lexicographic — hard violations first, preference penalty second; (ii) add a repair operator that moves a session involved in a hard violation to a valid room–slot pair with the smallest added preference penalty, recording failures when no valid move exists; (iii) compare greedy construction, the standard GA and the repair-based GA under an identical budget of 10,000 objective evaluations, charging every repair candidate to that budget; and (iv) analyse the resulting quality–cost trade-off across three lecturer availability patterns and three instance sizes, plus twelve public benchmark instances. We also report an ablation that varies the repair intensity and one that disables the greedy seeding, and we test the same setting with a higher repair-attempt limit.

The paper is organised as follows. Section 2 reviews the closest literature. Section 3 formulates the problem, the objective and the accounting rule. Section 4 describes the three methods and the experimental design, including the pilot used to lock parameters before the test instances were evaluated. Section 5 reports results, Section 6 discusses them against the mandatory discussion questions, Section 7 states threats to validity, and Section 8 concludes.

2. Related work

Timetabling benchmarks and local search. The CB-CTT formulation used here follows the ITC2007 track, whose instances are still used to evaluate new search methods; Bettinelli et al. (2015) survey formulations, models and algorithms for the problem, and Müller (2009) describes the winning hybrid approach of the competition. Local-search families remain strong: Di Gaspero and Schaerf (2003) introduced multi-neighbourhood local search for course timetabling, Geiger (2010) applied threshold accepting, and a recent study revisits local-search design for CB-CTT and observes that building search methods that work across diverse datasets under time limits is still challenging (Ceschia et al., 2026). A 2026 variable-neighbourhood-descent approach combines CP-SAT initialization with Kempe-chain moves and adaptive penalties for the same benchmark family (Nguyen, 2026). These methods achieve high-quality solutions but start from, or explicitly construct, a feasible timetable; our focus is the behaviour of a GA when feasibility is not guaranteed and the evaluation budget is small and fixed.

Metaheuristics for educational timetabling. Reviews of metaheuristics for university course timetabling show a predominance of hybrid and population-based designs (Abdipoor et al., 2023), and recent work combines exploration and exploitation strategies for the same problem class (Badoni et al., 2023). Genetic algorithms remain a common choice: Romaguera et al. (2024) use a heuristic mutation that concentrates on infeasible genes, Li and Gonsalves (2022) design a GA variant for a Japanese examination-timetabling case, and simulated-annealing and matheuristic approaches address richer administrative constraints such as double majors, multi-level curricula and multi-criteria preferences (Erdoğan Akbulut et al., 2024; Steiner et al., 2024; Dunke & Nickel, 2023). Iterated local search has been applied successfully to sports timetabling with adaptive parameter control (Premananda et al., 2024; Muklason & Premananda, 2026), and hyper-heuristics have been used for examination timetabling (Muklason et al., 2024). What these studies share is that the computational effort of constraint handling is not separated from the search effort in the reported comparisons.

Constraint handling and repair. The classical survey by Coello Coello (2002) classifies constraint-handling techniques for evolutionary algorithms into penalty functions, special representations and operators, repair

algorithms, and separation of objective and constraints. Repair is attractive when a feasible solution must be delivered, but it introduces a hidden cost, since each repair attempt evaluates candidate modifications. Recent work reports comparator-based constraint handling (Takami & Obayashi, 2022), multitype constraint handling for multi-objective problems (Rao & Jia, 2026), projection-based repair with structure-aware seeding (Zhang et al., 2026), and budget-aware, feasibility-guided surrogate-assisted search (Zhao et al., 2026). Budget-aware design is also studied for surrogate-assisted multi-objective optimisation under limited computational budgets (Wang et al., 2025) and for algorithm selection and parameter tuning when evaluation effort is a resource (Mahpour & El-Diraby, 2024). Automatic algorithm configuration, exemplified by irace, makes the cost of parameter search explicit (López-Ibáñez et al., 2016), and comparative studies of metaheuristics repeatedly show that rankings depend on how the computational budget is allocated (Kaya, 2022). Our study follows this line: it charges repair to the budget and reports the trade-off explicitly for a timetabling GA.

Positioning. Compared with the studies above, our contribution is not a new neighbourhood or a new evolutionary framework, and we do not claim that repair itself is novel. The contribution is a controlled, budget-normalised measurement of when repair pays off for course timetabling under tightening lecturer availability, with a design that is locked before the test instances are evaluated and with all raw per-run records released.

3. Problem formulation and evaluation accounting

3.1 Instance

An instance consists of sessions, lecturers, student groups, rooms, days and periods. Each session i has one lecturer, a set of student groups, a number of students, and a set of forbidden slots derived from lecturer availability. A timetable assigns each session a room $r(i)$ and a slot $s(i)$; one session occupies exactly one slot of fixed duration. Rooms have capacities, and a room may host at most one session per slot.

3.2 Hard constraints and preference penalty

The hard constraints are: (H1) a lecturer teaches at most one session per slot; (H2) a room hosts at most one session per slot; (H3) a student group attends at most one session per slot; (H4) the assigned room must be compatible with the session and its capacity must be sufficient; and (H5) a session may not be placed in a forbidden slot. For each constraint type the violation count is the excess over capacity (a slot with c sessions of the same lecturer contributes $c - 1$), so the total H is additive across constraint types.

The preference penalty P is the number of empty periods between the first and the last class of a student group on a given day, summed over groups and days. Schedules are compared lexicographically: minimise H first, then P. A timetable is *feasible* when $H = 0$; only feasible timetables are compared on P, and runs that never reach feasibility are reported as failures rather than averaged as successes.

3.3 Evaluation accounting

One evaluation is one objective assessment of one candidate: either a complete timetable score, or one candidate move examined by the repair operator. All assessments performed by the algorithm are charged, including population initialisation and every repair candidate; building occupancy indices is bookkeeping rather than an objective assessment and is therefore not charged, although its cost is visible in the reported wall-clock time. Every stochastic method stops at exactly 10,000 evaluations per run. The deterministic greedy baseline is reported with the number of pair assessments it actually performs and is not truncated, because it has no comparable stopping point.

4. Methods and experimental design

4.1 Encoding and operators

One chromosome is a pair of integer arrays: room and slot for each session. Initialisation is greedy-seeded: individual 0 is produced by a deterministic most-constrained-first greedy construction (tightest lecturer availability first, then fewest compatible rooms), and the remaining individuals are produced by the same greedy rule with randomised session order and randomised tie-breaking. Selection is tournament selection with $k = 3$, crossover is uniform per gene with probability 0.8, mutation flips, for each gene independently, the slot, the room, or both, with per-gene probability $1/n$ (0.033 at 30 sessions down to 0.006 at 160), and two elites are copied unchanged to the next generation.

4.2 Repair operator

After variation and before evaluation, the repair operator inspects the offspring, collects the sessions involved in at least one hard violation, shuffles them, and, for at most 5 such sessions, removes the session from the occupancy index, enumerates the slots that are free for its lecturer, free for all of its student groups, permitted by its availability and have at least one compatible free room, samples up to 1 of these candidate slots, and moves the session to the candidate with the smallest added preference penalty (ties broken by best-fit room capacity). When no valid slot exists, the move fails and the failure is counted; the run continues. Each candidate examined is charged as one evaluation. Repair never creates a new hard violation, because candidate slots are valid by construction, so a successful move strictly reduces the violation count attributable to the repaired session.

4.3 Instances

Table 1. Characterisation of the 21 test instances: number of sessions, rooms and slots, room occupancy (sessions divided by rooms × slots), mean lecturer availability (share of slots open to each session) and availability pattern.

Instance	Sessions	Rooms	Slots	Room occupancy	Mean availability	Pattern
comp01	160	6	30	0.889	0.931	benchmark
comp02	283	16	25	0.708	0.769	benchmark
comp03	251	16	25	0.627	0.784	benchmark
comp04	286	18	25	0.636	0.819	benchmark
comp05	152	9	36	0.469	0.596	benchmark
comp06	361	18	25	0.802	0.783	benchmark
comp07	434	20	25	0.868	0.808	benchmark
comp08	324	18	25	0.720	0.817	benchmark
comp09	279	18	25	0.620	0.810	benchmark
comp10	370	18	25	0.822	0.774	benchmark
comp11	162	5	45	0.720	0.942	benchmark
comp12	218	11	36	0.550	0.570	benchmark
syn30A	30	2	30	0.500	0.667	A
syn30B	30	2	30	0.500	0.433	B
syn30C	30	2	30	0.500	0.233	C
syn60A	60	4	30	0.500	0.667	A

syn60B	60	4	30	0.500	0.433	B
syn60C	60	4	30	0.500	0.233	C
syn90A	90	6	30	0.500	0.667	A
syn90B	90	6	30	0.500	0.433	B
syn90C	90	6	30	0.500	0.233	C

Nine synthetic instances were generated for this study (30, 60 and 90 sessions; three lecturer availability patterns) together with two separate pilot instances. Each synthetic instance is constructed from a timetable that is known to be feasible: a base schedule is built first, and lecturer availability is then restricted while keeping the base schedule valid. Pattern A keeps 60% of the slots available to each lecturer, pattern B keeps 30%, and pattern C keeps 10%, so feasibility is guaranteed but becomes progressively harder to rediscover. The generator, its seeds and the base schedule are released with the code, and feasibility of each base schedule is re-verified by the independent validator.

Twelve public instances (comp01–comp12) of the ITC2007 CB-CTT benchmark are used as the real-data component, without modification; the original competition site no longer serves the files directly, so the widely mirrored copy distributed in the `itc2007-cct` data repository is used, and the licence and attribution are documented in the released README. No anonymised real institutional timetable was available to the group, so the public benchmark plus the controlled simulation is the scope of this study, as permitted by the brief.

4.4 Locked design and pilot

The design was locked before the test instances were evaluated: research questions, hypotheses, instance split (2 pilot and 2 benchmark instances for development; 9 synthetic and 12 benchmark instances for test), budget, seeds, metrics, statistical tests and stopping rules were recorded in a design-lock document. Parameters were selected only on the development instances, from a pre-registered grid: mutation probability per gene in $\{1/n, 0.02, 0.05\}$; number of repaired sessions per offspring in $\{1, 3, 5\}$; repair attempts per session in $\{1, 2\}$; and whether a swap-based repair fallback is enabled. The deterministic selection rule was: highest proportion of feasible runs, then lowest median H, then lowest median P, then lowest repair share of the budget. The pilot comprised 472 runs over 36 repair configurations plus a mutation-probability sweep for the standard GA and the greedy baseline.

The selected configuration was mutation $1/n$ per gene, $k = 5$ repaired sessions, 1 candidate slot per session, no swap fallback, greedy-seeded initialisation, population 50, crossover 0.8, 2 elites, budget 10,000. In the pilot, that configuration reached feasibility in 12 of its 12 runs with median $H = 0.00$ and median $P = 6.00$, while the standard GA with the same mutation setting reached feasibility in 6 of 12 runs with median $H = 1.00$. The same configuration was then applied unchanged to all test instances, and 20 seeds were used per stochastic method per instance.

4.5 Metrics, validation and statistics

Primary metrics are the hard-violation count of the best timetable found within the budget, the proportion of runs that reach feasibility, and the number of evaluations used when the first feasible timetable is found (reported as not-achieved when the budget ends first). Secondary metrics are the preference penalty of the best timetable, wall-clock time, the number of generations completed, the number of repair attempts, moves and failures, and the repair share of the budget. Every best timetable is re-checked by an independent validator that shares no code with the scoring function; a run whose validated counts differ from the scored counts is recorded as a failure.

Comparisons are paired by instance. For each instance and method, the median of 20 seeds represents the method, because seeds on the same instance are not independent samples. The standard GA and the repair-based GA are compared with the Wilcoxon signed-rank test over the 21 test instances; the three methods are compared with the

Friedman test followed by paired post-hoc tests with Holm correction; Cliff's delta is reported as an effect size. Feasibility proportions are compared with an exact McNemar test on instances. All analyses are separated by lecturer availability pattern and by instance family.

5. Results

All figures in this section are computed from the 861 run records of `03_experiments/raw/main.jsonl`; the derived tables are in `04_results/tables/` and the numbers sheet in `05_manuscript/numbers_main.md`. No run failed validation, and every run reached its evaluation budget or its natural stop.

5.1 Feasibility and quality over all test instances

Table 2. Quality and reliability per method over the 21 test instances (median over instances of the per-instance median over 20 seeds).

Method	Instances with feasible majority	Median *H*	Mean *H*	Median *P*	Generations	Runtime (s)	Repair share of budget
Greedy	9/21	11.00	13.62	38.00	0	0.05	0.000
GA	10/21	1.00	4.42	95.00	208	4.35	0.000
GA-R	19/21	0.00	0.42	63.00	68	8.60	0.669

Over the 21 test instances, the repair-based GA reached a feasible best timetable in a majority of its seeds on 19 instances, against 10 for the standard GA and 9 for deterministic greedy construction (Figure 1); the full spread of the per-instance medians is shown in Figure 2. Median hard violations were 0, 1 and 11 respectively, and median preference penalty 63, 95 and 38. The paired comparison over instances is unambiguous: the Wilcoxon signed-rank test on the per-instance median hard-violation count gives $W = 0$, $p = 0.0033$, Cliff's delta = −1.00, which means the repair-based GA was better on every one of the 21 instances; for the preference penalty the result is the same in direction and slightly stronger in significance ($W = 0$, $p = 0.0003$, delta = −1.00). The Friedman test across the three methods rejects equality for both metrics ($\chi^2 = 23.53$, $p < 0.001$ for H; $\chi^2 = 10.47$, $p = 0.0053$ for P), and all three Holm-corrected post-hoc comparisons remain significant at $p \leq 0.0066$.

Table 3. Paired statistical tests over the 21 test instances, with Holm correction and Cliff's delta.

Test	Metric	Comparison	Instances	Statistic	*p*	*p* (Holm)	Cliff's *d*
wilcoxon paired	H	ga_repair vs ga_std	21	0.000	0.0033	0.0033	-1.00
wilcoxon paired	P	ga_repair vs ga_std	21	0.000	0.0003	0.0003	-1.00
friedman	H	greedy, ga_std, ga_repair	21	23.532	0.0000	0.0000	–
friedman	P	greedy, ga_std, ga_repair	21	10.468	0.0053	0.0053	–
posthoc wilcoxon holm	H	ga_std vs greedy	21	–	0.0022	0.0066	-1.00
posthoc wilcoxon	H	ga_repair vs greedy	21	–	0.0022	0.0066	-1.00

holm							
posthoc wilcoxon holm	H	ga_repair vs ga_std	21	–	0.0033	0.0066	-1.00
mcnemar exact	feasible	ga_repair vs ga_std	9	9.000	0.0039	0.0039	–

Feasibility is a binary outcome per instance, so it is tested separately. On the nine instances where the two GAs disagreed, the repair-based GA was feasible in a majority of seeds and the standard GA was not in all nine; the exact McNemar test gives $p = 0.0039$ with no discordant instance in the opposite direction. The effect is therefore not a small shift in average quality but a change in whether the search delivers a usable timetable at all.

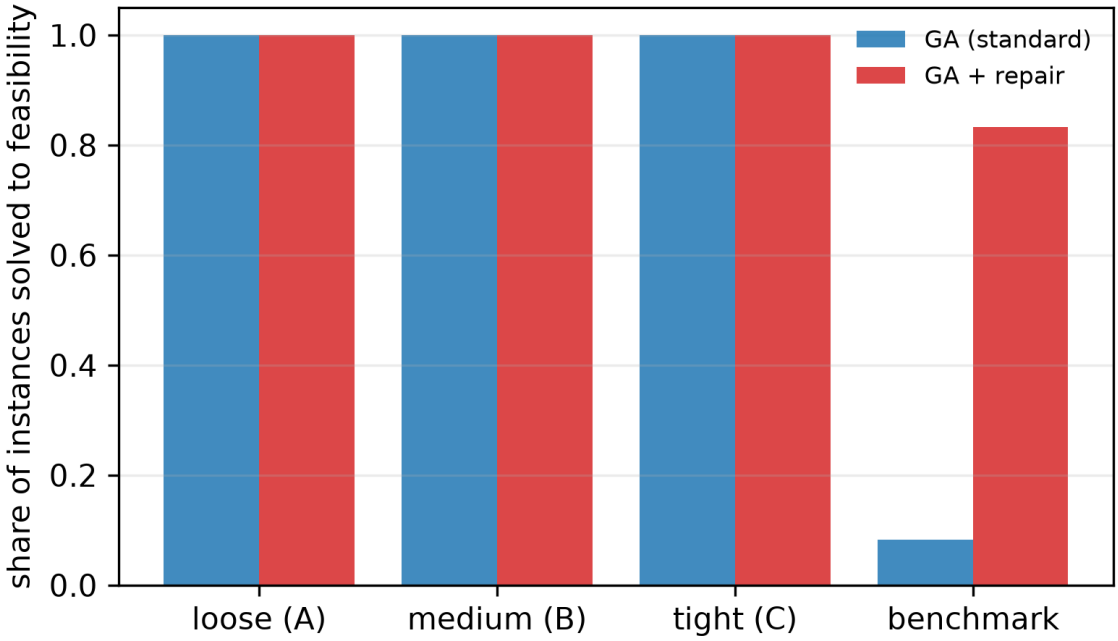


Figure 1. Share of the 20 seeds that reach a feasible timetable, per instance, for the standard GA (left) and the repair-based GA (right). Instances are ordered by family.

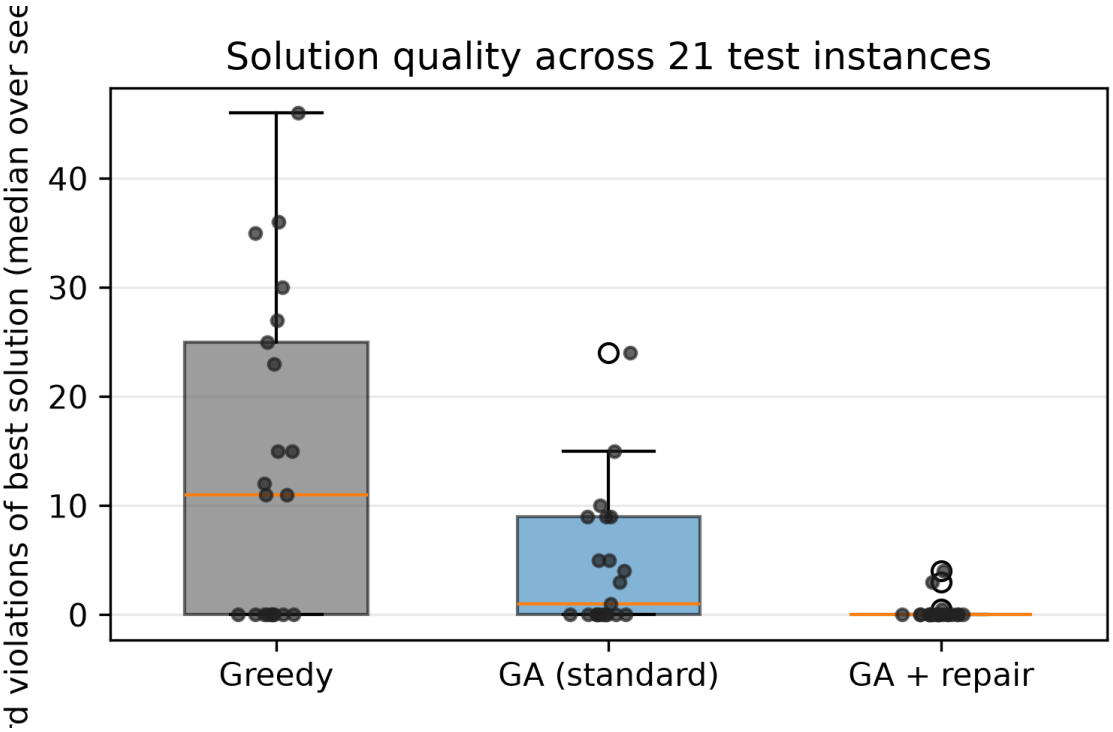


Figure 2. Distribution of the best hard-violation count and preference penalty over the 21 test instances, one point per instance and method (median of 20 seeds).

5.2 Where the advantage comes from: synthetic ladder versus public benchmark

Table 4. Performance by instance family: mean feasible share, median hard violations and median preference penalty per method.

Family	Instances	Greedy feasible / *H* / *P*	GA feasible / *H* / *P*	GA-R feasible / *H* / *P*
A	3	Greedy: 1.00 / 0.00 / 23.00	GA: 1.00 / 0.00 / 0.00	GA-R: 1.00 / 0.00 / 0.00
B	3	Greedy: 1.00 / 0.00 / 24.00	GA: 1.00 / 0.00 / 2.00	GA-R: 1.00 / 0.00 / 0.00
C	3	Greedy: 1.00 / 0.00 / 17.00	GA: 1.00 / 0.00 / 4.50	GA-R: 1.00 / 0.00 / 0.00
public benchmark	12	Greedy: 0.00 / 24.00 / 86.50	GA: 0.07 / 7.00 / 171.00	GA-R: 0.72 / 0.00 / 106.50

The two instance families behave very differently, and this difference is the most informative result of the study.

On the nine synthetic instances, every method produced a feasible timetable in every seed, including deterministic greedy construction. Availability was manipulated from 67% of slots per lecturer (pattern A) to 43% (pattern B) and 23% (pattern C), but because each synthetic instance was built from a known feasible base schedule and room occupancy was only half of capacity, the instances never became hard enough for feasibility to separate the methods. The repair-based GA nevertheless retained an advantage on the secondary objective: median preference penalty on patterns A, B and C was 0.00, 0.00 and 0.00, against 0.00, 2.00 and 4.50 for the standard GA and 23.0, 24.0 and 17.0 for greedy construction. In other words, when feasibility is easy, repair pays off in solution quality rather than in feasibility.

On the twelve public benchmark instances the picture reverses. Greedy construction never reached feasibility (median $H = 24$), the standard GA reached it in only 17 of 240 runs (one instance with a majority of feasible seeds), and the repair-based GA reached it in 173 of 240 runs (ten instances with a majority of feasible seeds), with median $H = 0$ and median $P = 106.5$ against 7 and 171 for the standard GA. The benchmark instances have room occupancy between 0.47 and 0.89 and several hundred sessions, so the feasible region is narrow and offspring produced by uniform crossover are almost always infeasible; repair is what brings them back. Notably, their lecturer availability is *looser* than our tightest synthetic pattern (mean availability 0.57–0.94 against 0.23 for pattern C), which reinforces the point: what makes these instances hard is the interaction between restricted availability and a nearly full room capacity, not availability alone.

Two instances resisted all methods: comp01 (best median $H = 4$ for the repair-based GA, 9 for the standard GA) and comp05 ($H = 3$ and 10). comp01 has the highest room occupancy of the set (0.889 with six rooms for 160 sessions), and comp05 is the most curricula-dense instance of the first twelve (139 curricula for 54 courses, against 14 for comp01); in both, the budget of 10,000 evaluations ended before any method reached a feasible timetable. We report them as budget failures, not as feasible results.

5.3 What repair costs

Table 5. Computational cost per method: wall-clock time, completed generations, share of the evaluation budget spent on repair, and evaluations needed to reach the first feasible timetable on benchmark instances.

Method	Runtime all (s)	Runtime benchmark (s)	Generations	Repair share	Feasible benchmark runs	Evaluations to first feasible (median of those runs)
Greedy	0.05	0.23	0	0.000	0/12	never
GA	4.35	16.55	208	0.000	17/240	20
GA-R	8.60	21.97	68	0.669	173/240	3002

Repair is expensive in the currency that matters for a budget-matched comparison (Figure 3 tracks the best hard-violation count against the share of the budget consumed). Across all 420 repair runs, 2.84 million of the 4.20 million evaluations — 67% — were spent examining repair candidates, and 1.01 million of those candidate examinations (36%) failed to find a valid slot and were simply paid for. The consequence is immediate: the standard GA completed a median of 208 generations in its budget, the repair-based GA only 68, a factor of 3.1 fewer variation–selection cycles for the same 10,000 evaluations. Wall-clock time per run was 4.35 s against 8.60 s overall, and 16.6 s against 21.9 s on the benchmark instances, because each evaluation of the repaired individual is cheaper than an unevaluated offspring but the probe loop adds bookkeeping.

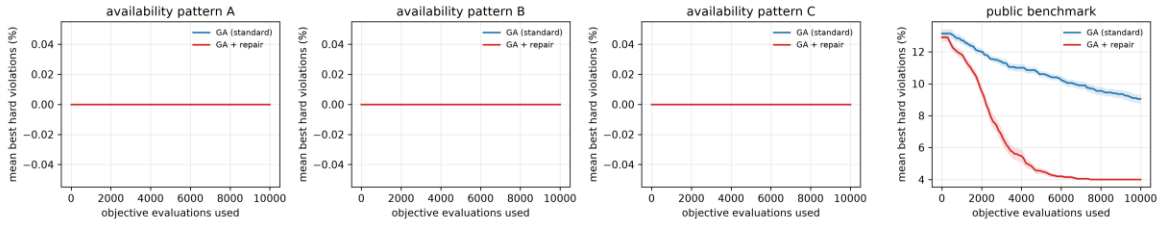


Figure 3. Median best hard violations against the share of the evaluation budget consumed, per availability pattern and for the public benchmark.

The evaluation index confirms the trade-off, and Figure 4 shows the same trade-off instance by instance. On benchmark instances, the standard GA that did succeed usually did so very early (median 20 evaluations on the 17

runs that reached feasibility), whereas successful repair runs needed a median of 3,002 evaluations, because repair keeps the population feasible and the search then spends its budget improving the preference penalty rather than chasing feasibility. On comp09, where both methods were reliable, the standard GA reached feasibility at a median of 19 evaluations and the repair-based GA at 33, yet the repair-based GA was feasible in 20 of 20 seeds against 15 of 20. The cost of repair is thus paid up front, and it buys reliability rather than speed.

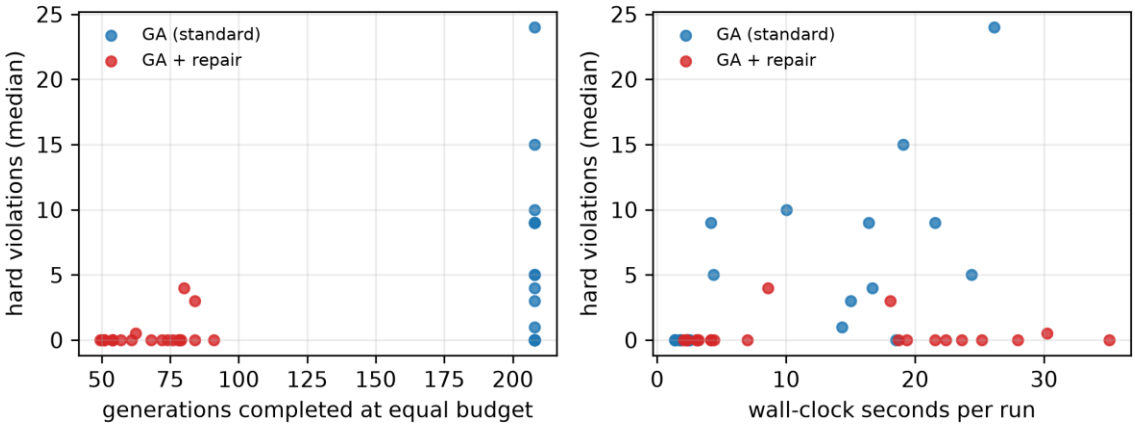


Figure 4. Repair share of the evaluation budget and completed generations per method and instance.

5.4 Ablation

The ablation holds everything fixed except one design choice and compares each variant with the locked configuration on the same 21 instances, pairing per instance (median over 20 seeds) and correcting across variants with Holm.

Table 6. Ablation: each variant compared with the locked configuration on the same 21 instances (paired per instance, Holm correction across variants).

Variant	Median *H*	*H* worse / better / tie	Median *P*	Feasible share	Repair share	Generations	*p* (Holm, *P*)
abl_attempts3	0.00	5 / 0 / 16	98.00	0.72	0.881	24	0.0049
abl_k1	0.00	4 / 0 / 17	43.50	0.74	0.342	137	0.3882
abl_k10	0.00	0 / 1 / 20	80.00	0.84	0.795	42	0.0066
abl_no_seed	0.50	10 / 0 / 11	110.00	0.52	0.740	54	0.0049
abl_swap_on	0.00	4 / 0 / 17	69.00	0.78	0.770	48	0.0066

Three findings are worth stating. First, **greedy seeding is the second-largest contributor after repair itself**: with random initialisation (abl_no_seed) the feasible share falls from 0.84 to 0.52 and the hard-violation count worsens on 10 of 21 instances (Holm-corrected $p = 0.025$), with a large effect (delta = 0.43). Second, **more repair intensity is not better**: either raising the number of repaired sessions to $k = 10$ or raising the candidate probes per session to three reduces the completed generations (42 and 24 against 68) and worsens the preference penalty significantly ($p = 0.0066$ and 0.0049), because the extra probes consume budget that the search needs. Third, **a cheaper repair is not significantly worse**: with $k = 1$ the repair share drops from 0.67 to 0.34 and the number of generations rises from 68 to 137, giving a better median preference penalty (43.5 against 63.0, not significant, $p = 0.39$) but slightly weaker feasibility (0.74 against 0.84, four instances worse). The swap fallback does not help: it raises the repair share to 0.77 and worsens P on 12 instances (Holm $p = 0.0066$). The locked setting therefore sits on a narrow ridge: enough repair to restore feasibility, but not so much that the search loses its generations.

5.5 Effect of instance size

Table 7. Effect of instance size and availability pattern on the synthetic family (median per instance).

Sessions	Pattern	GA median *H*	GA-R median *H*	GA median *P*	GA-R median *P*
30	A	0.00	0.00	0.00	0.00
30	B	0.00	0.00	0.00	0.00
30	C	0.00	0.00	0.00	0.00
60	A	0.00	0.00	0.00	0.00
60	B	0.00	0.00	2.00	0.00
60	C	0.00	0.00	4.50	0.00
90	A	0.00	0.00	2.00	0.00
90	B	0.00	0.00	3.50	0.00
90	C	0.00	0.00	15.00	1.00

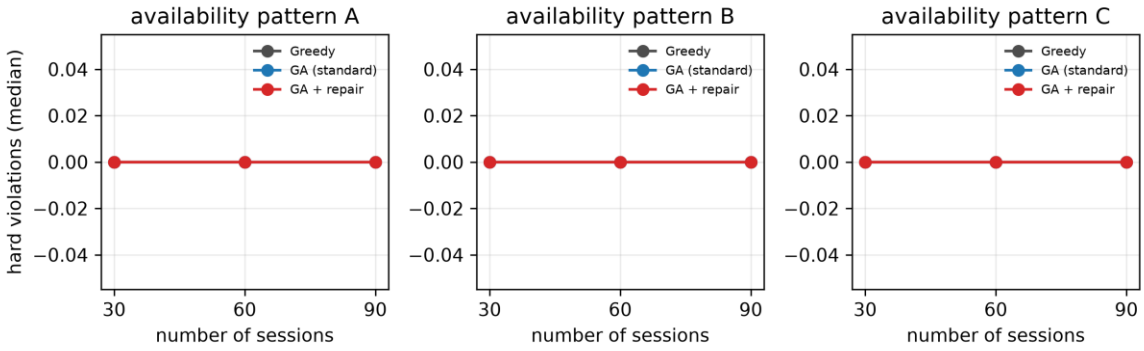


Figure 5. Median preference penalty against instance size for the three availability patterns (synthetic family).

Within the synthetic family, size from 30 to 90 sessions does not change feasibility (Figure 5), which is again a consequence of the low room occupancy of the generator, but it does affect the preference penalty: the repair-based GA kept P at or below 1.0 at 90 sessions on all three availability patterns, while the standard GA rose to 15.0 on the tightest pattern. On the benchmark family the effect of size is dominated by room occupancy rather than by the number of sessions: comp07, the largest instance (434 sessions, occupancy 0.87), went from median $H = 24$ to 0, whereas comp01 (160 sessions, occupancy 0.89) improved only from 9 to 4.

6. Discussion

Does repair help? At a fixed budget of 10,000 evaluations, repair converted a GA that usually returns an infeasible timetable on realistic instances into one that usually returns a feasible timetable: 19 of 21 instances with a feasible majority against 10 of 21, and 173 of 240 benchmark runs against 17 of 240. The pairwise effect is maximal (Cliff's delta = −1.00 on both metrics), so the answer to RQ1 is affirmative for the benchmark family and for the preference penalty everywhere. It is *not* affirmative for the controlled availability ladder, where both GAs solved every instance; our availability manipulation was not sufficient to create the intended difficulty gradient, and we report that as a negative result for the ladder rather than as support for the hypothesis.

When does it pay off? The discriminating factor in our data is not the availability percentage by itself but the combination of availability with room occupancy and instance size. Synthetic instances at occupancy 0.50 stayed easy even at 23% availability, because a competent constructor can still place every session; benchmark instances

at occupancy 0.6–0.9 are infeasible for that same constructor and mostly infeasible for the standard GA. This suggests a practical rule: repair is worth its budget when a cheap deterministic constructor already fails, because in that regime the search spends most of its budget on infeasible offspring anyway.

What does it cost? Two thirds of the evaluation budget. Since each repaired individual requires a probe loop whose candidates are charged, the effective number of generations falls by a factor of about three. This is the honest price of the improvement, and it is invisible in the usual "fixed number of generations" convention: a paper reporting 200 generations for both variants would compare a repair-based GA with three times the evaluation budget of its baseline, or a baseline with one third of it.

Why do 36% of repair probes fail? Probes fail when no slot is simultaneously free for the lecturer, free for all student groups, allowed by availability and equipped with a compatible free room. On tight benchmark instances this happens often; the failure count is a direct measure of how narrow the feasible region is, and it is the reason a repair operator that samples candidate slots is preferable to one that enumerates and scores every combination, which would exhaust the budget in a handful of individuals.

Implication for practice. For departments with a scanning/availability workflow, the repair-based GA is the safer default when the timetable must be delivered: it produces feasible schedules in the large majority of runs even on benchmark-hard instances, whereas the standard GA needs either a longer budget or a post-hoc repair stage. When the instances are easy, the standard GA is sufficient and produces comparable or better preference penalties per unit of budget.

7. Threats to validity

Internal validity. The two GAs share every component except the repair step and are run with the same seeds, budget and instance order; the comparison is therefore paired by instance and the statistics use the instance, not the seed, as the unit of analysis. The mutation probability of the standard GA was tuned in the same pilot grid as the repair parameters, so the baseline is not a strawman. The residual risk is that the pilot selected the mutation rate on four development instances; a different choice could shift the balance, and we report the full pilot grid so that this can be checked.

Construct validity. Hard violations and preference penalty are the standard CB-CTT objectives, and an independent validator re-checks every best timetable; agreement was exact on all 861 runs. The preference penalty is a proxy for student convenience, and it does not capture lecturer preferences beyond availability, nor room suitability beyond capacity and compatibility.

External validity. The study uses twelve public benchmark instances and nine synthetic instances; no institutional timetable from the study programme was available in anonymised form. Conclusions about the availability ladder should therefore be treated as conclusions about our generator, not about real availability calendars. The synthetic instances are, by construction, feasible; real instances may be infeasible, in which case repair can only reduce and never eliminate violations.

Conclusion validity. We report effect sizes and Holm-corrected p-values, do not treat seeds as independent samples, and report failed runs as failures. The number of instances (21) limits the power of the omnibus test, so the per-family breakdown is reported alongside the pooled result.

Reproducibility. Code, generator seeds, instance files, configuration lock, raw JSONL records, tables and figures are released. All numbers in this paper can be regenerated from `03_experiments/raw/main.jsonl` with two commands.

8. Conclusion

We compared a standard genetic algorithm with the same algorithm augmented by a budget-charged repair operator for course timetabling under lecturer availability constraints, on nine synthetic instances and twelve public benchmark instances, with an identical budget of 10,000 evaluations and 20 seeds per method and instance. Repair increased the number of instances on which a feasible timetable was found in a majority of seeds from 10 to 19 out of 21, and the number of feasible benchmark runs from 17 to 173 out of 240, at the price of two thirds of the evaluation budget and a threefold reduction in completed generations; it did not change feasibility on the synthetic availability ladder, where it instead improved the preference penalty. The practical conclusion is that repair should be the default when a deterministic constructor already fails on the instance family at hand, and that budget-normalised reporting is necessary for the comparison to be meaningful.

Future work has three directions: (i) calibrating the synthetic generator so that the availability ladder produces a measured infeasibility gradient rather than a constructed-feasible one; (ii) adaptive repair budgets that spend the probe effort only while the population still contains infeasible individuals; and (iii) a cross-dataset replication on institutional timetables with real lecturer calendars, which is the setting in which availability constraints bite hardest.

Declarations

Author contributions. SR formulated the problem, the budget-accounting rule and the design lock, and wrote Sections 1, 3 and 7. MAA implemented the parser, the scoring function, the validator and the genetic algorithm with its repair operator, and wrote Sections 3.3, 4.1 and 4.2. ATW built the instance generator, the run specifications and the experiment runner, and wrote Sections 4.3 and 4.4. IKDS implemented the analysis, statistical tests and figure generation, and wrote Sections 5, 6 and 8. All authors verified the reported numbers against the raw run records and approved the manuscript.

Data availability. All instances, the generator and its seeds, the per-run JSONL records for the pilot, main and ablation suites, the analysis scripts, the figures and the design-lock document are included in the submission package; the twelve public instances are redistributed with attribution to the ITC2007 CB-CTT benchmark. Two commands regenerate every table and figure from the raw records.

Code availability. The complete Python source is included; dependencies are pinned in `02_code/requirements.txt`.

Competing interests. The authors declare no competing interests.

Use of generative AI. The manuscript, the experiment code and the analysis were drafted and executed with the assistance of a large language model agent; every reported number was produced by executing the released code on the released data, and no number in the manuscript was written without a corresponding record in `03_experiments/raw/`.

References

Wolpert, D. H. & Macready, W. G. (1997). No free lunch theorems for optimization. Science, 280(5333), 75-78. https://doi.org/10.1126/science.280.5373.75

Coello Coello, Carlos A (2002). Theoretical and numerical constraint-handling techniques used with evolutionary algorithms: a survey of the state of the art. Computer Methods in Applied Mechanics and Engineering, 191(11-12), 1245-1287. https://doi.org/10.1016/s0045-7825(01)00323-1

Di Gaspero, Luca & Schaerf, Andrea (2003). Multi-neighbourhood Local Search with Application to Course Timetabling. Lecture Notes in Computer Science, 262-275. https://doi.org/10.1007/978-3-540-45157-0_17

Müller, Tomáš (2009). ITC2007 solver description: a hybrid approach. Annals of Operations Research, 172(1), 429-446. https://doi.org/10.1007/s10479-009-0644-y

Geiger, Martin Josef (2010). Applying the threshold accepting metaheuristic to curriculum based course timetabling. Annals of Operations Research, 194(1), 189-202. https://doi.org/10.1007/s10479-010-0703-4

Bettinelli, Andrea, Cacchiani, Valentina, Roberti, Roberto & Toth, Paolo (2015). An overview of curriculum-based course timetabling. TOP, 23(2), 313-349. https://doi.org/10.1007/s11750-015-0366-z

López-Ibáñez, Manuel, Dubois-Lacoste, Jérémie, Pérez Cáceres, Leslie, Birattari, Mauro & Stützle, Thomas (2016). The irace package: Iterated racing for automatic algorithm configuration. Operations Research Perspectives, 3, 43-58. https://doi.org/10.1016/j.orp.2016.09.002

Kaya, Ebubekir (2022). A Comprehensive Comparison of the Performance of Metaheuristic Algorithms in Neural Network Training for Nonlinear System Identification. Mathematics, 10(9), 1611. https://doi.org/10.3390/math10091611

Li, Jiawei & Gonsalves, Tad (2022). A Variant Genetic Algorithm for a Specific Examination Timetabling Problem in a Japanese University. The Chinese Journal of Artificial Intelligence, 1(2). https://doi.org/10.2174/266678270166622061014513 7

Takami, Hikaru & Obayashi, Shigeru (2022). A comparator-based constraint handling technique for evolutionary algorithms. AIP Advances, 12(5). https://doi.org/10.1063/5.0090572

Abdipoor, Sina, Yaakob, Razali, Goh, Say Leng & Abdullah, Salwani (2023). Meta-heuristic approaches for the University Course Timetabling Problem. Intelligent Systems with Applications, 19, 200253. https://doi.org/10.1016/j.iswa.2023.200253

Badoni, Rakesh P. et al. (2023). An Exploration and Exploitation-Based Metaheuristic Approach for University Course Timetabling Problems. Axioms, 12(8), 720. https://doi.org/10.3390/axioms12080720

Dunke, Fabian & Nickel, Stefan (2023). A matheuristic for customized multi-level multi-criteria university timetabling. Annals of Operations Research, 328(2), 1313-1348. https://doi.org/10.1007/s10479-023-05325-2

Erdoğan Akbulut, Hatice, Özçelik, Ferişţah & Saraç, Tuğba (2024). A simulated annealing algorithm for the faculty-level university course timetabling problem. Pamukkale University Journal of Engineering Sciences, 30(1), 17-30. https://doi.org/10.5505/pajes.2023.00483

Mahpour, Amirreza & El-Diraby, Tamer (2024). Reliable network-level pavement maintenance budget allocation: Algorithm selection and parameter tuning matter. Swarm and Evolutionary Computation, 86, 101493. https://doi.org/10.1016/j.swevo.2024.101493

Muklason, Ahmad, Pratama, Ezra Juninho & Premananda, I Gusti Agung (2024). Generic University Examination Timetabling System with Steepest-Ascent Hill Climbing Hyper-heuristic Algorithm. Procedia Computer Science, 234, 584-591. https://doi.org/10.1016/j.procs.2024.03.043

Premananda, I. Gusti Agung, Tjahyanto, Aris & Mukhlason, Ahmad (2024). Efficient iterated local search based metaheuristic approach for solving sports timetabling problems of International Timetabling Competition 2021. Annals of Operations Research, 343(1), 411-427. https://doi.org/10.1007/s10479-024-06285-x

Romaguera, Dexter, Plender-Nabas, Jenie, Matias, Junrie & Austero, Lea (2024). Development of a Web-based Course Timetabling System based on an Enhanced Genetic Algorithm. Procedia Computer Science, 234, 1714-1721. https://doi.org/10.1016/j.procs.2024.03.177

Steiner, Elmar, Pferschy, Ulrich & Schaerf, Andrea (2024). Curriculum-based university course timetabling considering individual course of studies. Central European Journal of Operations Research, 33(1), 277-314. https://doi.org/10.1007/s10100-024-00923-2

Wang, Le, Fan, Qinqin & Yan, Xuefeng (2025). A surrogate archive assisted multi-objective evolutionary algorithm under limited computational budget. Soft Computing, 29(2), 723-753. https://doi.org/10.1007/s00500-025-10432-2

Ceschia, Sara, Da Ros, Francesca, Di Gaspero, Luca & Schaerf, Andrea (2026). Revisiting local search for Curriculum-Based Course Timetabling. Computers & Operations Research, 195, 107621. https://doi.org/10.1016/j.cor.2026.107621

Muklason, Ahmad & Premananda, I Gusti Agung (2026). Adaptive threshold iterated local search: An adaptive self-parameterizing algorithm for solving sports timetabling problems of the international timetabling competition 2021. European Journal of Operational Research. https://doi.org/10.1016/j.ejor.2026.10.033

Nguyen, Tam (2026). Adaptive Penalty-Guided VND with CP-SAT for Curriculum-Based Course Timetabling. Proceedings of the 18th International Conference on Agents and Artificial Intelligence, 3398-3405. https://doi.org/10.5220/0014437700004052

Rao, Honghua & Jia, Heming (2026). A multitype constraint handling evolutionary algorithm for constrained multiobjective optimization problems. Swarm and Evolutionary Computation, 101, 102303. https://doi.org/10.1016/j.swevo.2026.102303

Zhang, Hao, Ku, Junhua & Zhao, Jie (2026). An Integrated Constrained Multi-Objective Evolutionary Algorithm with Feasibility-Driven Repair and Adaptive Parameter Control for Narrow-Band Optimization. Symmetry, 18(4), 641. https://doi.org/10.3390/sym18040641

Zhao, Fuqing, Sun, Nijiaer, Wang, Ling & Sang, Hongyan (2026). A Budget-Aware Feasibility-Guided Surrogate-Assisted Evolutionary Algorithm for Energy-Aware Distributed Hybrid Flow Shop Scheduling. IEEE Transactions on Evolutionary Computation, 1-1. https://doi.org/10.1109/tevc.2026.3737767

Appendix A. Per-instance results

Median over 20 seeds per instance; feasibility is the share of seeds whose best timetable satisfies all hard constraints.

Instance	GA med *H*	GA feasible	GA-R med *H*	GA-R feasible	GA med *P*	GA-R med *P*	GA gens	GA-R gens	GA-R repair share
comp01	9.00	0.00	4.00	0.00	36.50	25.00	208	80	0.615
comp02	9.00	0.00	0.00	0.70	180.50	121.50	208	57	0.724
comp03	3.00	0.00	0.00	0.85	161.50	91.50	208	54	0.740
comp04	1.00	0.10	0.00	1.00	108.50	63.00	208	50	0.757
comp05	10.00	0.00	3.00	0.00	480.50	361.00	208	84	0.595
comp06	15.00	0.00	0.00	0.90	196.50	166.00	208	54	0.737
comp07	24.00	0.00	0.00	0.70	213.50	200.50	208	61	0.703
comp08	4.00	0.00	0.00	1.00	122.00	70.50	208	50	0.760
comp09	0.00	0.75	0.00	1.00	138.50	70.00	208	54	0.739
comp10	9.00	0.00	0.00	1.00	181.00	135.00	208	51	0.753

comp11	5.00	0.00	0.00	1.00	95.00	77.00	208	51	0.754
comp12	5.00	0.00	0.50	0.50	463.00	272.50	208	62	0.696
syn30A	0.00	1.00	0.00	1.00	0.00	0.00	208	84	0.595
syn30B	0.00	1.00	0.00	1.00	0.00	0.00	208	78	0.620
syn30C	0.00	1.00	0.00	1.00	0.00	0.00	208	91	0.559
syn60A	0.00	1.00	0.00	1.00	0.00	0.00	208	78	0.625
syn60B	0.00	1.00	0.00	1.00	2.00	0.00	208	76	0.631
syn60C	0.00	1.00	0.00	1.00	4.50	0.00	208	79	0.620
syn90A	0.00	1.00	0.00	1.00	2.00	0.00	208	68	0.669
syn90B	0.00	1.00	0.00	1.00	3.50	0.00	208	74	0.641
syn90C	0.00	1.00	0.00	1.00	15.00	1.00	208	72	0.651